Performance inspection

C2 and OC on separate computers

Run an optional monitor beside each node, view both computers in one desktop dashboard, and keep local performance logs for later review. This guide assumes Ubuntu with ROS 2 Jazzy and your existing, working C2-to-OC network configuration.

C2 computer	Shared ROS network	OC computer
C2 GUI + local monitor Performance dashboard	Same domain and rover namespace Performance topic: /rover1/inspection/metrics	OC + local monitor USB board or PTY simulator

1. Update and build on BOTH computers

For an existing checkout, stop its nodes before rebuilding. Preserve any local edits before pulling; do not discard them if Git reports a conflict.

```
cd ~/weeny_ws
git pull --ff-only origin main
source /opt/ros/jazzy/setup.bash
rosdep install --from-paths src --ignore-src -r -y
./scripts/build.sh
source install/setup.bash
```

For a new computer, clone first, then run the build commands above:

```
git clone \
  https://github.com/CU-URC-2027/Distributed-Computers-v1.git \
  ~/weeny_ws
```

2. Prepare EVERY terminal used below

```
source /opt/ros/jazzy/setup.bash
source ~/weeny_ws/install/setup.bash
```

Apply the same ROS_DOMAIN_ID, discovery/static-peer settings, and middleware that already work between your computers. Use the same namespace on both; examples use rover1. The new launchers inherit your settings. The older run_demo.sh, run_hardware.sh, and dcv1 tools still impose their existing local configuration.

Additional dependencies: python3-psutil and python3-tk. The dashboard and C2 GUI need a graphical desktop. Prepared November 5, 2026. Instructions: pages 2-3; dashboard: page 4; logs/settings: page 5; troubleshooting: page 6.

Run with the hardware OC

Use this page for the OC connected to your verified v2 Arduino. For a simulator, follow page 3 instead. Start only one OC for a given serial port and rover namespace.

1. On the OC computer

Find the stable USB device path, then replace YOUR_VERIFIED_BOARD with the correct entry. Use your established board and motor commissioning procedure.

```
ls -l /dev/serial/by-id/

ros2 launch mini_rover_oc oc.launch.py namespace:=rover1 \
  port:=/dev/serial/by-id/YOUR_VERIFIED_BOARD \
  inspect_performance:=true
```

This starts OC and its local monitor. Hardware mode is the default. It does not start a C2 GUI on the OC computer. The monitor records locally even if the network connection drops.

2. On the C2 computer

```
ros2 launch mini_rover_c2 c2.launch.py namespace:=rover1 \
  inspect_performance:=true inspection_dashboard:=true
```

Two windows open: the existing C2 control GUI and the performance dashboard. The dashboard should show C2 and OC, with their actual hostnames and process IDs. Allow a few seconds for ROS discovery.

3. Confirm observation before operating

Keep the controller disabled while checking the display. Confirm both cards show **LIVE**, the OC reports the expected hardware provenance, and the local log files are growing on both computers. C2 intent traffic can be absent before you claim control; that is normal.

Continue using the existing C2 Connect/claim, neutral Enable, and hold-to-drive controls when ready. The performance dashboard only observes the system.

Stop or turn inspection off

Press Ctrl+C in each launch terminal to stop its processes. Closing the C2 control GUI ends its launch session and companions. Closing only the performance window leaves control running. A monitor failure also leaves the control node running.

```
# Start C2 without performance inspection:
ros2 launch mini_rover_c2 c2.launch.py \
  inspect_performance:=false
```

For OC, keep the required port argument and set inspect_performance:=false. Inspection is off by default. ROS launch uses name:=value syntax, rather than --inspect-performance.

Simulation or existing node sessions

The simulation route uses the same two-computer network and dashboard, with a PTY simulator running on the OC computer. No physical Arduino is required.

Simulation: OC computer, terminal A

```
ros2 run mini_rover_sim arduino_sim \
  --port-link /tmp/rover-inspection-arduino
```

Leave this terminal running. The simulator prints its PTY path. If this path is already active, use the existing simulator or choose a new path consistently; do not start a duplicate.

Simulation: OC computer, terminal B

```
ros2 launch mini_rover_oc oc.launch.py namespace:=rover1 \
  port:=/tmp/rover-inspection-arduino simulated:=true \
  inspect_performance:=true
```

On the C2 computer, use the C2 command from page 2. The dashboard will label chassis data **SIMULATION**. The simulator must be running before launching OC. Stop OC and C2 with Ctrl+C, then stop the simulator in terminal A.

Attach to an already running node

Keep your current startup commands and run a standalone monitor on each computer. Use the actual local c2_gui or oc process ID from your process manager, not the ros2 launch wrapper PID. The number 12345 below is an example to replace.

```
ros2 run mini_rover_inspection performance_monitor \
  --role oc --pid 12345 --namespace /rover1 --interval 1 \
  --log-directory ~/weeny_ws/performance_logs
```

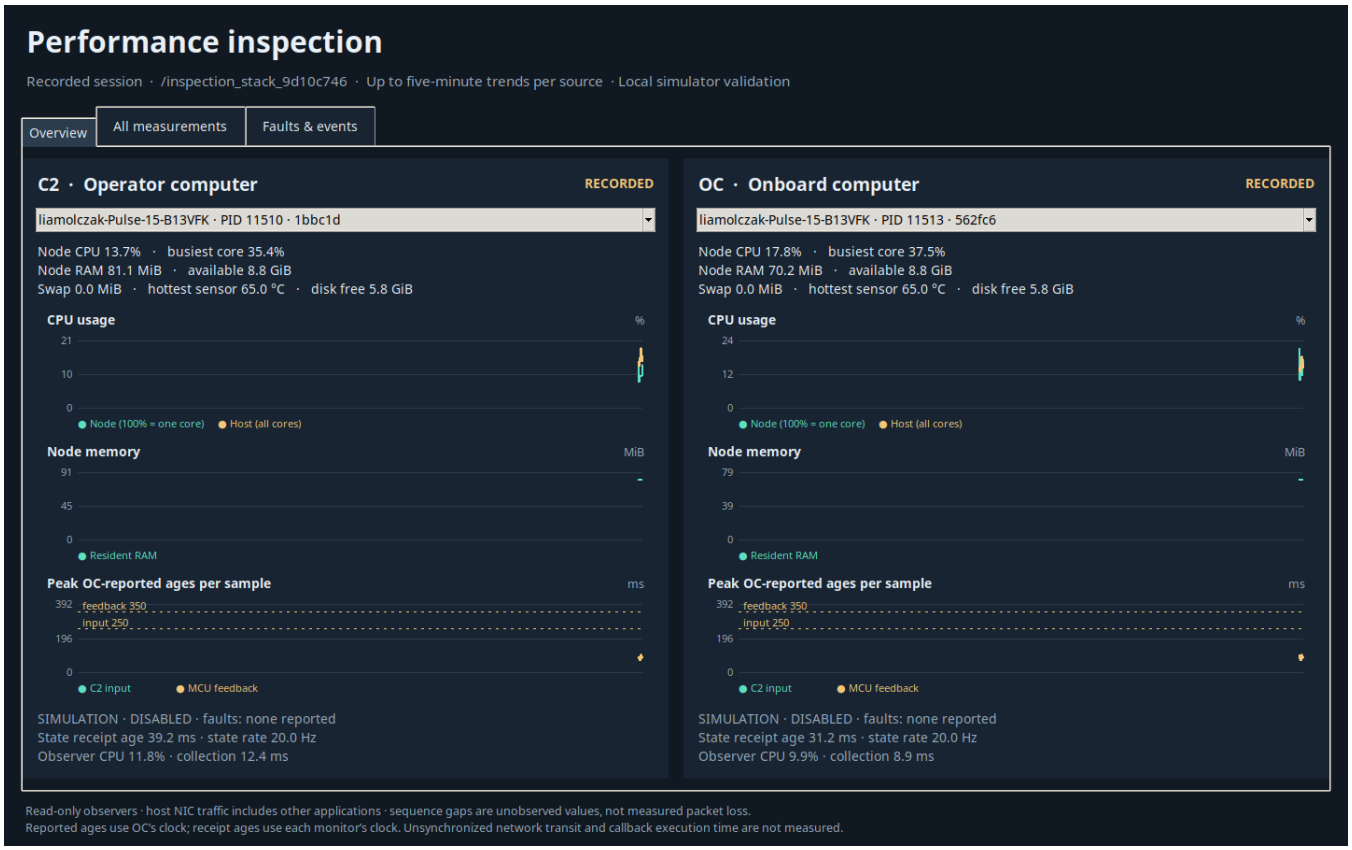
On the C2 computer, use --role c2 and its local C2 process ID. Each monitor tracks that process and exits when it exits. Restart a manually attached monitor after restarting its node.

Open the dashboard independently

```
ros2 run mini_rover_inspection performance_dashboard \
  --namespace /rover1
```

Run the viewer on the C2 computer, or another computer with the same ROS environment and a graphical desktop. Standalone monitors stop with Ctrl+C without stopping the node they observe.

Read the dashboard



Example: actual local simulator samples, explicitly shown as a recorded session. Both roles ran on one laptop for this verification; your deployment should display its two hostnames.

Area	What it tells you
Overview	CPU/RAM trends; busiest core; free memory/disk; temperature; OC input and MCU feedback ages; controller state; monitor overhead.
All measurements	NIC and disk throughput, interface errors/drops, swap, sensor age, process threads and file descriptors, topic rates/gaps, serial diagnostics, and logging status.
Faults & events	Fault appearances/clearances, identity changes, serial errors, and process exits captured between resource samples.

LIVE means samples are arriving. **STALE** means none arrived for the larger of three seconds or three sample intervals; shown values are last known. **EXITED** means target termination was reported. **RECORDED** means you are viewing a saved session.

Process CPU uses 100% for one fully occupied core and can exceed 100%. Host CPU averages all cores. RAM is the process resident memory. N/A means unavailable, not zero. Select a source at the top of a card if multiple sessions exist.

Charts show up to five minutes per source, limited to 600 retained samples (two minutes at 0.2-second sampling). Input-age peaks are omitted when C2 has no control session.

Settings, local logs, and replay

Launch setting	Default / purpose
inspect_performance	false; enable the local companion monitor.
inspection_dashboard	false; open the viewer here when inspection is also enabled.
inspect_interval	1.0 seconds; allowed range 0.2-60 seconds.
inspect_log_directory	~/weeny_ws/performance_logs; directory on this computer.
namespace	rover1; must match between the participants.

Example: sample twice per second on C2

```
ros2 launch mini_rover_c2 c2.launch.py \
  inspect_performance:=true inspection_dashboard:=true \
  inspect_interval:=0.5 \
  inspect_log_directory:=/home/YOUR_USER/rover-perf
```

Replace YOUR_USER, or omit inspect_log_directory to use the default. Changes take effect when you restart that launch. Temperature and CPU-frequency sensors are read in a background thread every five seconds regardless of the resource interval.

Find the recordings on EACH computer

```
ls -lt ~/weeny_ws/performance_logs/
```

Every run creates a unique c2- or oc-prefixed JSONL file. Each file rotates at 20 MiB and retains four backup parts, about 100 MiB per monitor run. Old runs are not automatically deleted. Archive or remove them deliberately when no longer needed. Log errors and dropped records appear in the dashboard; a performance logging failure does not change motor authority.

Open recorded data

Copy the OC files to the viewing computer using your normal file-transfer method. Replace these example paths with your actual files:

```
ros2 run mini_rover_inspection performance_dashboard --replay \
  /path/to/c2-run.jsonl /path/to/oc-run.jsonl
```

For rotated parts, pass the oldest first: .jsonl.4, .jsonl.3, .jsonl.2, .jsonl.1, then .jsonl. Replay loads the bounded recent history, labels it RECORDED, and does not connect to ROS. Invalid or truncated lines are skipped and counted.

Local logs keep samples through a network outage. Their wall timestamps can be correlated with the OC run logs, subject to the computers' clock synchronization error.

Checks and troubleshooting

Symptom	What to check
Package or executable missing	Pull/build the updated source on that computer and source install/setup.bash in the terminal. Verify python3-psutil and python3-tk are installed.
Only one host appears	Confirm inspection is enabled on both. Match namespace, ROS_DOMAIN_ID, discovery/static peers, middleware, and firewall settings with your working distributed setup.
No graphical window	Open the dashboard in a graphical desktop session. OC monitoring can run headlessly. A plain SSH shell may not have a display.
STALE card	Check the remote node and monitor processes, network connectivity, and remote local log. Stale performance data alone does not prove the control node failed.
No C2 intent traffic	An unclaimed C2 session is normally silent. Check state/chassis traffic first; monitoring itself never claims control.
OC simulation port rejected	Start the simulator first; simulated:=true requires an existing PTY or symlink to one. Hardware mode requires a /dev/serial/by-id/ path.
Log error / missing sensors	Check disk space and directory permissions. Unsupported temperature or frequency sensors legitimately show N/A.

Read-only ROS checks, in a prepared terminal

```
ros2 node list --no-daemon
ros2 topic info /rover1/inspection/metrics --verbose
ros2 topic hz /rover1/inspection/metrics
```

The topic combines all monitors: two monitors at a one-second interval normally contribute about two samples per second in total. It uses best-effort delivery; local JSONL is the source to inspect for samples absent from the remote viewer.

What these measurements do and do not establish

The 250 ms input and 350 ms feedback lines reflect the current control contract. Ages come from OC or the observer's local clock; they are not one-way network delay or physical stopping time. Sequence gaps mean unobserved values, not proven packet loss. NIC counters include other applications. Temperature/frequency do not independently prove thermal throttling.

Local verification passed 239 workspace Python tests plus C++ guard and 131 native/Python checks. The final sensor-thread change passed 57 focused tests, including real launchers, DDS diagnostics, recording, GUI rendering, and the PTY simulator. This feature still needs a check on your two physical computers; no remote hardware performance results are claimed.

Editable instructions: PERFORMANCE_INSPECTION.md in the repository. Rebuild this PDF with scripts/build_performance_guide.py using Python with reportlab and DejaVu fonts.